

5.7 Memento Pattern

Memento Pattern เป็นแพตเทิร์นพฤติกรรมที่ใช้ในการบันทึก และย้อนคืนสถานะ (state) ของออบเจกต์โดยไม่ละเมิดการห่อหุ้ม (encapsulation) เหมาะสำหรับสถานการณ์ที่ต้องการจัดการประวัติการเปลี่ยนแปลงของออบเจกต์ เช่น ระบบปรับแต่งบ้านอัจฉริยะที่สามารถย้อนคืนการตั้งค่าก่อนหน้าได้

:: *Memento Pattern* ::

บันทึกและย้อนคืนสถานะ (state) ของออบเจกต์

เหมาะสำหรับสถานการณ์ที่ต้องการจัดการประวัติการเปลี่ยนแปลงของออบเจกต์

ตัวอย่างปัญหา

ระบบจัดการสถานะบ้านบ้านอัจฉริยะ (Smart Home) ผู้ใช้อาจเปลี่ยนการตั้งค่าต่าง ๆ เช่น อุณหภูมิ ความสว่างของไฟ หรือโหมดการทำงาน (Work Mode) หากผู้ใช้เปลี่ยนใจ การย้อนคืนค่าก่อนหน้า

ตัวอย่าง Main program

```
SmartHome home = new SmartHome();
HouseHistory history = new HouseHistory();

// Initial state
home.setState("22°C", "Bright", "Work Mode");
history.save(home.saveState());

// Change state
home.setState("18°C", "Dim", "Relax Mode");
history.save(home.saveState()); // Save state

// Undo last change
System.out.println("\nUndoing last change...");
HouseState previousState = history.undo();
if (previousState != null) {
    home.restoreState(previousState);
}
```

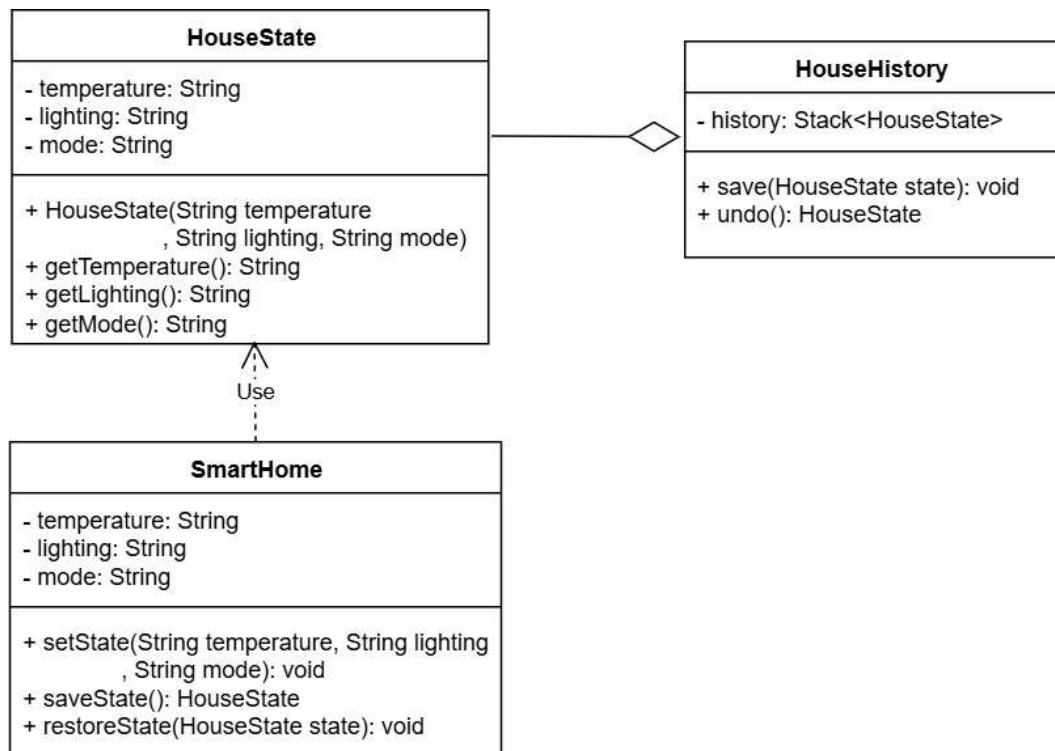
5.7.1 วัตถุประสงค์และตัวอย่าง

Memento Pattern ช่วยให้สามารถบันทึก (Capture) และกู้คืน (Restore) สถานะภายใน (Internal State) ของอ็อบเจกต์ได้โดยไม่เปิดเผยโครงสร้างภายใน ถูกใช้เพื่อจัดการฟังก์ชันที่เกี่ยวข้องกับการบันทึกและย้อนกลับสถานะ เช่น ฟังก์ชัน Undo

- 1) **การบันทึกและกู้คืนสถานะ (Saving and Restoring State)** โดยสร้างจุดบันทึก (Snapshot) ของสถานะปัจจุบันของอ็อบเจกต์ได้ตลอดเวลา และสามารถกลับไปสู่สถานะที่บันทึกไว้เหล่านั้นได้ ด้วยการสร้างอ็อบเจกต์ตัวแทนที่เรียกว่า Memento (ความทรงจำ) ซึ่งทำหน้าที่เก็บข้อมูลสถานะของอ็อบเจกต์หลัก (Originator) ณ เวลานั้น ๆ
- 2) **การรักษาการห่อหุ้มข้อมูล (Preserving Encapsulation)** หากต้องการบันทึกสถานะของอ็อบเจกต์ภายนอกจะต้องเข้าถึงข้อมูลภายในของมัน ซึ่งเป็นการละเมิดหลักการห่อหุ้ม (Encapsulation) โดย Originator (ผู้สร้าง) เท่านั้นที่สามารถเข้าถึงและแก้ไขสถานะภายในของตัวเองได้ ส่วน Memento จะเก็บสถานะไว้ แต่มีเพียง Originator เท่านั้นที่รู้ว่าต้องอ่านหรือเขียนข้อมูลอะไรจาก Memento Caretaker (ผู้ดูแล) ซึ่งเป็นผู้เก็บ Memento ไว้ (เช่น ปุ่ม Undo) จะเก็บ Memento ไว้เหมือนเป็น "กล่องดำ" และไม่สามารถเข้าถึงข้อมูลภายในได้เลย
- 3) **การรองรับฟังก์ชัน Undo/Rollback** เพื่อสร้างกลไกที่ยืดหยุ่นสำหรับฟังก์ชัน "ย้อนกลับ" (Undo) หรือการทำธุรกรรม (Transaction) โดยผู้ดูแล (Caretaker) สามารถเก็บ Memento หลายตัวไว้ในโครงสร้างข้อมูล (เช่น Stack) เมื่อผู้ใช้สั่ง Undo ผู้ดูแลจะดึง Memento ล่าสุดออกมา และส่งคืนให้ Originator เพื่อกู้คืนสถานะนั้น

ตัวอย่างการใช้งาน

ระบบจัดการสถานะบ้านบ้านอัจฉริยะ (Smart Home) ผู้ใช้อาจเปลี่ยนการตั้งค่าต่าง ๆ เช่น อุณหภูมิ ความสว่างของไฟ หรือโหมดการทำงาน (Work Mode) หากผู้ใช้เปลี่ยนใจ สามารถย้อนคืนค่าก่อนหน้าได้



ภาพที่ xx ระบบบ้านอัจฉริยะที่สามารถย้อนคืนค่าสถานะได้ โดยใช้ Memento Pattern

HouseState (Memento) เก็บสถานะของบ้าน เช่น อุณหภูมิ แสงไฟ โหมด

```

1. // Memento class: บันทึกสถานะ
2. class HouseState {
3.
4.     private String temperature;
5.     private String lighting;
6.     private String mode;
7.
8.     public HouseState(String temperature, String lighting
9.                        , String mode){
10.         this.temperature = temperature;
11.         this.lighting = lighting;
12.         this.mode = mode;
13.     }
14.
15.     public String getTemperature() {
16.         return temperature;
17.     }
18.
19.     public String getLighting() {
20.         return lighting;
21.     }
22.
23.     public String getMode() {
24.         return mode;
25.     }
  
```

26.}

SmartHome (Originator) มีหน้าที่บันทึกสถานะปัจจุบันและกู้คืนสถานะจาก Memento

```
1. // Originator class: บ้านอัจฉริยะ
2. class SmartHome {
3.
4.     private String temperature;
5.     private String lighting;
6.     private String mode;
7.
8.     public void setState(String temperature, String lighting
9.                           , String mode) {
10.         this.temperature = temperature;
11.         this.lighting = lighting;
12.         this.mode = mode;
13.         System.out.println("State updated: Temperature = "
14.                             + temperature + ", Lighting = " + lighting
15.                             + ", Mode = " + mode);
16.     }
17.
18.     public HouseState saveState() {
19.         return new HouseState(temperature, lighting, mode);
20.     }
21.
22.     public void restoreState(HouseState state) {
23.         this.temperature = state.getTemperature();
24.         this.lighting = state.getLighting();
25.         this.mode = state.getMode();
26.         System.out.println("State restored: Temperature = "
27.                             + temperature + ", Lighting = " + lighting
28.                             + ", Mode = " + mode);
29.     }
30. }
```

HouseHistory (Caretaker) เก็บประวัติของสถานะ (Memento) เพื่อรองรับการย้อนคืน

```
1. // Caretaker class: ดูแลประวัติของสถานะ
2. class HouseHistory {
3.
4.     private Stack<HouseState> history = new Stack<>();
5.
6.     public void save(HouseState state) {
7.         history.push(state);
8.     }
9.
10.    public HouseState undo() {
11.        if (!history.isEmpty()) {
12.            return history.pop();
13.        }
14.        return null;
15.    }
16. }
```

Main Class ใช้ HouseHistory เพื่อบันทึกสถานะ และย้อนคืนเมื่อจำเป็น

```
1. // Main class
2. public class MementoPatternExample {
3.     public static void main(String[] args) {
4.
5.         SmartHome home = new SmartHome();
6.         HouseHistory history = new HouseHistory();
7.
8.         // Initial state
9.         home.setState("22°C", "Bright", "Work Mode");
10.        history.save(home.saveState()); // Save state
11.
12.        // Change state
13.        home.setState("18°C", "Dim", "Relax Mode");
14.        history.save(home.saveState()); // Save state
15.
16.        // Change state again
17.        home.setState("25°C", "Bright", "Party Mode");
18.
19.        // Undo last change
20.        System.out.println("\nUndoing last change...");
21.        HouseState previousState = history.undo();
22.        if (previousState != null) {
23.            home.restoreState(previousState);
24.        }
25.
26.        // Undo another change
27.        System.out.println("\nUndoing another change...");
28.        previousState = history.undo();
29.        if (previousState != null) {
30.            home.restoreState(previousState);
31.        }
32.    }
33.}
```

ผลลัพธ์

State updated: Temperature = 22°C, Lighting = Bright, Mode = Work Mode

State updated: Temperature = 18°C, Lighting = Dim, Mode = Relax Mode

State updated: Temperature = 25°C, Lighting = Bright, Mode = Party Mode

Undoing last change...

State restored: Temperature = 18°C, Lighting = Dim, Mode = Relax Mode

Undoing another change...

State restored: Temperature = 22°C, Lighting = Bright, Mode = Work Mode

จะเห็นว่า Memento Pattern เหมาะสำหรับการบันทึกและกู้คืนสถานะของออบเจกต์ โดยในตัวอย่างนี้ช่วยให้ระบบบ้านอัจฉริยะสามารถบันทึกสถานะก่อนการเปลี่ยนแปลง ย้อนคืนสถานะเดิมได้ และลดความยุ่งยากและเพิ่มความสะดวกให้ผู้ใช้ในการจัดการสถานะบ้าน

5.7.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Memento Pattern ช่วยจัดการสถานะในระบบที่ต้องการ Undo/Redo และเพิ่มความสะดวกในการพัฒนาซอฟต์แวร์ที่ซับซ้อน ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 1 Text Editor (โปรแกรมแก้ไขข้อความ) ในโปรแกรมแก้ไขข้อความ เช่น Microsoft Word หรือ Google Docs เมื่อผู้ใช้พิมพ์ข้อความ โปรแกรมจะต้องสามารถย้อนกลับไปยังสถานะก่อนหน้าได้ เช่น การกดปุ่ม Undo

```
1. // Memento class
2. class TextState {
3.
4.     private String text;
5.
6.     public TextState(String text) {
7.         this.text = text;
8.     }
9.
10.    public String getText() {
11.        return text;
12.    }
13.}
```

```
1. // Originator class
2. class TextEditor {
3.
4.     private String text = "";
5.
6.     public void type(String newText) {
7.         text += newText;
8.         System.out.println("Current text: " + text);
9.     }
10.    public TextState save() {
11.        return new TextState(text);
12.    }
13.    public void restore(TextState state) {
14.        text = state.getText();
15.        System.out.println("Restored text: " + text);
16.    }
17.}
```

```

1. // Caretaker class
2. class TextHistory {
3.
4.     private Stack<TextState> history = new Stack<>();
5.
6.     public void save(TextState state) {
7.         history.push(state);
8.     }
9.
10.    public TextState undo() {
11.        return history.isEmpty() ? null : history.pop();
12.    }
13.}

```

```

1. // Main class
2. public class TextEditorExample {
3.     public static void main(String[] args) {
4.
5.         TextEditor editor = new TextEditor();
6.         TextHistory history = new TextHistory();
7.
8.         editor.type("Hello");
9.         history.save(editor.save());
10.
11.        editor.type(", World!");
12.        history.save(editor.save());
13.
14.        editor.type(" How are you?");
15.
16.        System.out.println("\nUndoing last change...");
17.        TextState previousState = history.undo();
18.        if (previousState != null) {
19.            editor.restore(previousState);
20.        }
21.
22.        System.out.println("\nUndoing another change...");
23.        previousState = history.undo();
24.        if (previousState != null) {
25.            editor.restore(previousState);
26.        }
27.    }
28.}

```

ผลลัพธ์

Current text: Hello

Current text: Hello, World!

Current text: Hello, World! How are you?

Undoing last change...

Restored text: Hello, World!

Undoing another change...

Restored text: Hello

ตัวอย่างที่ 1 ระบบเกมนับที่สถานะตัวละคร (Character Save State) ผู้เล่นสามารถบันทึกสถานะตัวละคร เช่น พลังชีวิต เลเวล และไอเท็ม เพื่อย้อนกลับมาเล่นต่อจากจุดที่บันทึกไว้

```
1. // Memento class
2. class CharacterState {
3.
4.     private int health;
5.     private int level;
6.
7.     public CharacterState(int health, int level) {
8.         this.health = health;
9.         this.level = level;
10.    }
11.
12.    public int getHealth() {
13.        return health;
14.    }
15.
16.    public int getLevel() {
17.        return level;
18.    }
19.}
```

```
1. // Originator class
2. class GameCharacter {
3.
4.     private int health;
5.     private int level;
6.
7.     public void setState(int health, int level) {
8.         this.health = health;
9.         this.level = level;
10.        System.out.println("Character State: Health = "
11.            + health + ", Level = " + level);
12.    }
13.
14.    public CharacterState saveState() {
15.        return new CharacterState(health, level);
16.    }
17.
18.    public void restoreState(CharacterState state) {
19.        this.health = state.getHealth();
20.        this.level = state.getLevel();
21.        System.out.println("Restored State: Health = "
```

```

22.             + health + ", Level = " + level);
23.     }
24.}

```

```

1. // Caretaker class
2. class GameHistory {
3.
4.     private Stack<CharacterState> history =
5.         new Stack<>();
6.
7.     public void save(CharacterState state) {
8.         history.push(state);
9.     }
10.
11.    public CharacterState undo() {
12.        return history.isEmpty() ? null : history.pop();
13.    }
14.}

```

```

1. // Main class
2. public class GameExample {
3.     public static void main(String[] args) {
4.
5.         GameCharacter character = new GameCharacter();
6.         GameHistory history = new GameHistory();
7.
8.         character.setState(100, 1);
9.         history.save(character.saveState());
10.
11.        character.setState(80, 2);
12.        history.save(character.saveState());
13.
14.        character.setState(50, 3);
15.
16.        System.out.println("\nUndoing last change...");
17.        CharacterState previousState = history.undo();
18.        if (previousState != null) {
19.            character.restoreState(previousState);
20.        }
21.    }
22.}

```

ผลลัพธ์

Character State: Health = 100, Level = 1

Character State: Health = 80, Level = 2

Character State: Health = 50, Level = 3

Undoing last change...

Restored State: Health = 80, Level = 2

ตัวอย่างที่ 2 ระบบวาดรูป (Drawing Application) ในแอปพลิเคชันวาดรูป ผู้ใช้สามารถวาดรูปร่างต่าง ๆ เช่น วงกลม สีเหลี่ยม และลบหรือย้อนกลับไปยังสถานะก่อนหน้าได้

```
1. // Memento class
2. class CanvasState {
3.
4.     private List<String> shapes;
5.
6.     public CanvasState(List<String> shapes) {
7.         this.shapes = new ArrayList<>(shapes);
8.     }
9.
10.    public List<String> getShapes() {
11.        return shapes;
12.    }
13.}
```

```
1. // Originator class
2. class Canvas {
3.
4.     private List<String> shapes = new ArrayList<>();
5.
6.     public void addShape(String shape) {
7.         shapes.add(shape);
8.         System.out.println("Added: " + shape
9.             + ". Current shapes: " + shapes);
10.    }
11.
12.    public CanvasState save() {
13.        return new CanvasState(shapes);
14.    }
15.
16.    public void restore(CanvasState state) {
17.        shapes = state.getShapes();
18.        System.out.println("Restored shapes: " + shapes);
19.    }
20.}
```

```
1. // Caretaker class
2. class CanvasHistory {
3.
4.     private Stack<CanvasState> history = new Stack<>();
5.
6.     public void save(CanvasState state) {
7.         history.push(state);
8.     }
9. }
```

```

8.     }
9.
10.    public CanvasState undo() {
11.        return history.isEmpty() ? null : history.pop();
12.    }
13.}

```

```

1. // Main class
2. public class DrawingApplication {
3.     public static void main(String[] args) {
4.
5.         Canvas canvas = new Canvas();
6.         CanvasHistory history = new CanvasHistory();
7.
8.         canvas.addShape("Circle");
9.         history.save(canvas.save());
10.
11.        canvas.addShape("Square");
12.        history.save(canvas.save());
13.
14.        canvas.addShape("Triangle");
15.
16.        System.out.println("\nUndoing last change...");
17.        CanvasState previousState = history.undo();
18.        if (previousState != null) {
19.            canvas.restore(previousState);
20.        }
21.    }
22.}

```

ผลลัพธ์

Added: Circle. Current shapes: [Circle]

Added: Square. Current shapes: [Circle, Square]

Added: Triangle. Current shapes: [Circle, Square, Triangle]

Undoing last change...

Restored shapes: [Circle, Square]

5.8 Mediator Pattern

Mediator Pattern เป็นแพตเทิร์นพฤติกรรมที่ช่วยจัดการการสื่อสารระหว่างวัตถุหลายตัว (objects) โดยบังคับให้สื่อสารผ่านตัวกลาง ทำให้แต่ละวัตถุไม่ต้องรับรู้ถึงรายละเอียดของการทำงานของวัตถุอื่นโดยตรง

:: *Mediator Pattern* ::

ช่วยจัดการการสื่อสารระหว่างวัตถุหลายตัว (objects) โดยบังคับให้สื่อสารผ่านตัวกลาง ทำให้แต่ละวัตถุไม่ต้องรับรู้ถึงรายละเอียดของการทำงานของวัตถุอื่นโดยตรง

ตัวอย่างปัญหา

บ้านอัจฉริยะมีอุปกรณ์ต่าง ๆ เช่น ไฟ เครื่องปรับอากาศ และประตู ที่สามารถควบคุมได้ผ่านระบบกลาง (Central System)

ตัวอย่าง Main program

```
SmartHomeController mediator = new
SmartHomeController();

Light light = new Light(mediator);
AirConditioner airConditioner = new AirConditioner(mediator);
Door door = new Door(mediator);

mediator.setLight(light);
mediator.setAirConditioner(airConditioner);
mediator.setDoor(door);

// จำลองเหตุการณ์ที่อ็อบเจกต์สื่อสารกัน
door.open();
mediator.notify(light, "temperatureHigh");
```

5.8.1 วัตถุประสงค์และตัวอย่าง

Mediator Pattern ช่วยลดความซับซ้อนของการสื่อสารระหว่างอ็อบเจกต์หลาย ๆ ตัว โดยการจำกัดให้การสื่อสารทั้งหมดต้องผ่านอ็อบเจกต์คนกลาง (Mediator) รวมทั้งจัดการกับสถานการณ์ที่อ็อบเจกต์ในระบบมีจำนวนมาก และต้องมีการโต้ตอบกันอย่างซับซ้อน ซึ่งส่งผลให้เกิดการพึ่งพาอาศัยกันสูง (Tight Coupling)

- 1) การลดการพึ่งพาระหว่างกัน (Decoupling) เมื่อระบบมีอ็อบเจกต์จำนวนมาก เรียกว่า Colleague ที่จำเป็นต้องสื่อสารกัน การอนุญาตให้แต่ละอ็อบเจกต์สื่อสารกันตรง ๆ ทำให้โค้ดยุ่งเหยิงและแก้ไขยาก เนื่องจากการเปลี่ยนแปลงในอ็อบเจกต์หนึ่งจะส่งผลกระทบต่ออ็อบเจกต์อื่นที่เกี่ยวข้องทั้งหมด การใช้ Mediator pattern จะช่วยแยกการสื่อสารออกไปไว้ในคลาส Mediator ทำให้ Colleagues (อ็อบเจกต์ย่อย) รู้จักเพียง Mediator เท่านั้น และไม่รู้จักรังซึ่งกันและกัน เมื่อ Colleague หนึ่งต้องการสื่อสารกับอีก Colleague หนึ่ง ก็จะส่งข้อความไปยัง Mediator ซึ่งรับผิดชอบในการส่งต่อข้อความไปยัง Colleague ปลายทางที่ถูกต้อง
- 2) การรวมตรรกะการโต้ตอบไว้ที่เดียว (Centralizing Control Logic) นำตรรกะที่ควบคุมวิธีการโต้ตอบของอ็อบเจกต์ต่าง ๆ มารวมไว้ที่เดียว แทนที่จะกระจายตรรกะการสื่อสารไปทั่วทุกคลาส โดยมี Mediator เป็นศูนย์กลางในการกำหนดว่าเมื่อ Colleague A ทำสิ่งนี้แล้วจะเกิดอะไรขึ้นกับ Colleague B และ C
- 3) การเพิ่มความสามารถในการนำกลับมาใช้ใหม่ (Improving Reusability) เมื่อ Colleague ถูกแยกส่วนจากการโต้ตอบที่ซับซ้อน เพราะารู้จักแค่ Mediator จึงสามารถนำไปใช้ในบริบทอื่น ๆ ที่มีการสื่อสารแตกต่างกันได้ง่ายขึ้น

ตัวอย่างที่ค่อนข้างชัดเจน เช่น ในระบบการควบคุมการจราจรทางอากาศ (Air Traffic Control)

- Colleagues คือ เครื่องบินแต่ละลำ
- Mediator คือ หอควบคุม (Control Tower)

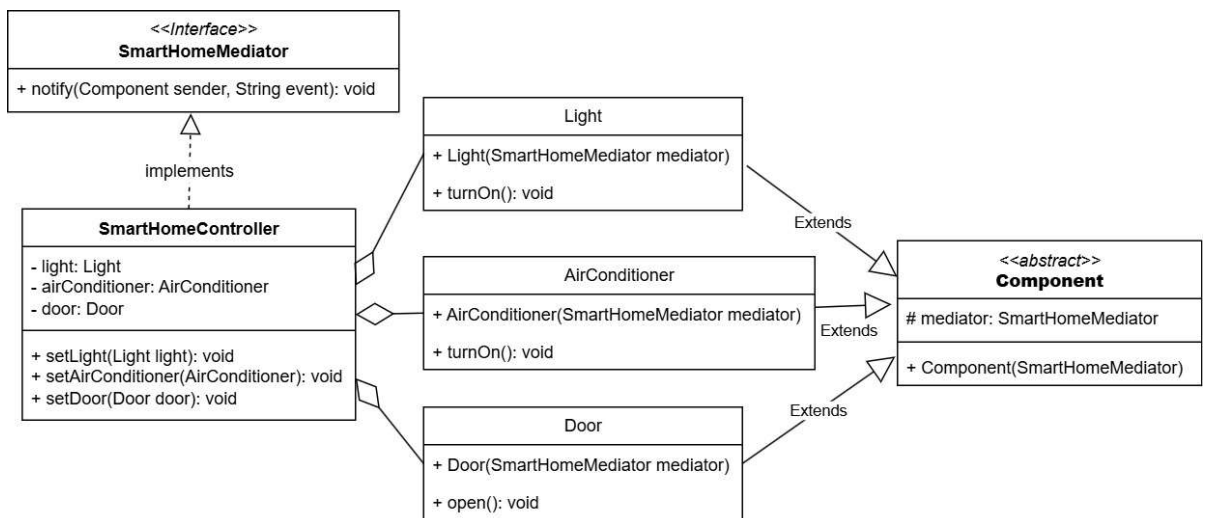
เครื่องบินลำหนึ่งไม่จำเป็นต้องรู้ว่าเครื่องบินลำอื่น ๆ บินอยู่ที่ไหน แต่เครื่องบินแต่ละลำจะสื่อสารกับหอควบคุมเท่านั้น และหอควบคุมจะเป็นผู้ประสานงานและส่งคำสั่งกลับไปให้เครื่องบินแต่ละลำ ทำให้เครื่องบินมีความเป็นอิสระต่อกันสูง

เหตุผลที่ควรใช้ Mediator Pattern

- ลดความซับซ้อนของการขึ้นต่อกัน (Coupling) เพราะว่าการที่แต่ละวัตถุสื่อสารกันโดยตรง ทำให้มีความสัมพันธ์ที่ยุ่งเหยิงและยากต่อการบำรุงรักษา
- แยกความรับผิดชอบ วัตถุแต่ละตัวจะรับผิดชอบเฉพาะหน้าที่ของตัวเอง และการสื่อสารจะถูกจัดการโดย Mediator
- การเพิ่มความสามารถในการขยายระบบ สามารถเพิ่มหรือเปลี่ยนแปลงวัตถุได้โดยไม่กระทบกับวัตถุอื่นมากนัก

ตัวอย่างการใช้งาน

ตัวอย่างบ้านอัจฉริยะมีอุปกรณ์ต่าง ๆ เช่น ไฟ (Light), เครื่องปรับอากาศ (Air Conditioner) และประตู (Door) ที่สามารถควบคุมได้ผ่านระบบกลาง (Central System) โดยใช้ Mediator Pattern เพื่อจัดการการสื่อสารระหว่างวัตถุที่เป็นอุปกรณ์ต่าง ๆ ในบ้านอัจฉริยะผ่านตัวกลางที่ชื่อ SmartHomeController



ภาพที่ xx ระบบบ้านอัจฉริยะที่ควบคุมด้วยการสื่อสารผ่านระบบกลาง โดยใช้ Mediator Pattern

Interface Mediator (`SmartHomeMediator`) กำหนดพฤติกรรมร่วมให้กับตัวกลาง (Mediator) ที่ต้องรองรับการแจ้งเตือนจากอุปกรณ์ต่าง ๆ

```
1. // Mediator Interface
2. interface SmartHomeMediator {
3.     void notify(Component sender, String event);
4. }
```

Concrete Mediator (`SmartHomeController`) เป็นตัวกลางจริงที่จัดการการทำงานระหว่างวัตถุ เช่น ถ้าประตูเปิด Door จะส่งสัญญาณ `doorOpened` แล้ว Mediator จะสั่งให้ไฟเปิด (`Light.turnOn()`)

```
1. // Concrete Mediator
2. class SmartHomeController implements SmartHomeMediator {
3.
4.     private Light light;
5.     private AirConditioner airConditioner;
```

```

6.     private Door door;
7.
8.     public void setLight(Light light) {
9.         this.light = light;
10.    }
11.
12.    public void setAirConditioner(
13.        AirConditioner airConditioner) {
14.        this.airConditioner = airConditioner;
15.    }
16.
17.    public void setDoor(Door door) {
18.        this.door = door;
19.    }
20.
21.    @Override
22.    public void notify(Component sender, String event) {
23.        if (event.equals("doorOpened")) {
24.            System.out.println("Door opened.
25.                Turning on the light...");
26.            light.turnOn();
27.        } else if (event.equals("temperatureHigh")) {
28.            System.out.println("Temperature is high.
29.                Turning on the air conditioner...");
30.            airConditioner.turnOn();
31.        }
32.    }
33.}

```

Abstract Component กำหนดโครงสร้างพื้นฐานให้กับอุปกรณ์ต่าง ๆ ที่ทำงานร่วมกับ Mediator

```

1. // Abstract Component
2. abstract class Component {
3.
4.     protected SmartHomeMediator mediator;
5.
6.     public Component(SmartHomeMediator mediator) {
7.         this.mediator = mediator;
8.     }
9. }

```

Concrete Components อุปกรณ์แต่ละตัว เช่น Light, AirConditioner และ Door ที่มีการสื่อสารผ่าน Mediator เช่น Door.open() จะเรียก mediator.notify() เพื่อแจ้งเหตุการณ์

```

1. // Concrete Components
2. class Light extends Component {
3.     public Light(SmartHomeMediator mediator) {
4.         super(mediator);
5.     }
6. }

```

```

7.     public void turnOn() {
8.         System.out.println("Light is turned on.");
9.     }
10.}

```

```

1. class AirConditioner extends Component {
2.
3.     public AirConditioner(SmartHomeMediator mediator) {
4.         super(mediator);
5.     }
6.
7.     public void turnOn() {
8.         System.out.println("Air conditioner is turned on.");
9.     }
10.}

```

```

1. class Door extends Component {
2.
3.     public Door(SmartHomeMediator mediator) {
4.         super(mediator);
5.     }
6.
7.     public void open() {
8.         System.out.println("Door is opened.");
9.         mediator.notify(this, "doorOpened");
10.    }
11.}

```

Main Class สร้างออบเจกต์ของ Mediator และอุปกรณ์ จากนั้นกำหนดความสัมพันธ์ระหว่างกัน และทดลองทำงาน

```

1. // Main class
2. public class MediatorExample {
3.     public static void main(String[] args) {
4.
5.         SmartHomeController mediator = new
6.             SmartHomeController();
7.
8.         Light light = new Light(mediator);
9.         AirConditioner airConditioner = new
10.             AirConditioner(mediator);
11.         Door door = new Door(mediator);
12.
13.         mediator.setLight(light);
14.         mediator.setAirConditioner(airConditioner);
15.         mediator.setDoor(door);
16.
17.         // Simulate events
18.         door.open();
19.         mediator.notify(light, "temperatureHigh");
20.     }
21.}

```

ผลลัพธ์

Door is opened.

Door opened. Turning on the light...

Light is turned on.

Temperature is high. Turning on the air conditioner...

Air conditioner is turned on.

ผลลัพธ์

- เมื่อ `Door.open()` ถูกเรียก
- ประตูกำลังเหตุการณ์ `doorOpened` ให้กับ `SmartHomeController`
- `SmartHomeController` แจ้งให้ Light เปิดไฟ
 - ข้อความ "Door opened. Turning on the light..." และ "Light is turned on." จะแสดงในผลลัพธ์
- เมื่อ `mediator.notify(light, "temperatureHigh")` ถูกเรียก
- Mediator สั่งให้ `AirConditioner` เปิดเครื่องปรับอากาศ
 - ข้อความ "Temperature is high. Turning on the air conditioner..." และ "Air conditioner is turned on." จะแสดงในผลลัพธ์

5.8.2 ตัวอย่างการประยุกต์ใช้ในการทำงานจริง

Mediator Pattern ช่วยลด Coupling และเพิ่มความสามารถในการขยายระบบได้ดี ดังตัวอย่างต่อไปนี้

ตัวอย่างที่ 1 ระบบแชท (Chat Application) ในระบบแชท ผู้ใช้หลายคนสามารถสื่อสารกันได้ในห้องสนทนาเดียวกัน โดย Mediator Pattern สามารถใช้เป็นตัวกลางในการจัดการข้อความระหว่างผู้ใช้ เพื่อให้แต่ละผู้ใช้ไม่ต้องสื่อสารกันโดยตรง และเพิ่มความสามารถในการจัดการระบบได้ง่ายขึ้น

หากผู้ใช้แต่ละคนต้องส่งข้อความโดยตรงถึงผู้อื่น จะเกิดปัญหา Coupling ระหว่างวัตถุ ทำให้โค้ดยุ่งเหยิงและขยายได้ยาก ดังนั้น การใช้ Mediator เป็นตัวกลางให้ทุกคนสื่อสารผ่านระบบกลางแทน

Mediator ChatRoom ทำหน้าที่เป็นตัวกลางที่รับและส่งข้อความระหว่างผู้ใช้ จัดเก็บรายชื่อผู้ใช้ทั้งหมดในห้องแชท

```
1. // Mediator Interface
2. interface ChatMediator {
3.     void sendMessage(String message, User sender);
4.     void addUser(User user);
5. }

1. // Concrete Mediator
2. class ChatRoom implements ChatMediator {
3.
4.     private List<User> users = new ArrayList<>();
5.
6.     @Override
7.     public void sendMessage(String message
8.                             , User sender) {
9.         for (User user : users) {
10.            // ไม่ส่งข้อความกลับไปให้ผู้ส่งเอง
11.            if (user != sender) {
12.                user.receive(message, sender.getName());
13.            }
14.        }
15.    }
16.
17.    @Override
18.    public void addUser(User user) {
19.        users.add(user);
20.    }
21.}

1. // Abstract User
2. abstract class User {
3.
4.     protected ChatMediator mediator;
5.     protected String name;
6.
7.     public User(ChatMediator mediator, String name) {
8.         this.mediator = mediator;
9.         this.name = name;
10.    }
11.
12.    public String getName() {
13.        return name;
14.    }
15.
16.    public abstract void send(String message);
17.    public abstract void receive(String message
18.                                , String senderName);
19.}
```

ChatUser ส่งข้อความผ่าน ChatRoom โดยที่ผู้ใช้แต่ละคนไม่รู้รายละเอียดของกันและกัน

```
1. // Concrete User
2. class ChatUser extends User {
3.
4.     public ChatUser(ChatMediator mediator, String name) {
5.         super(mediator, name);
6.     }
7.
8.     @Override
9.     public void send(String message) {
10.        System.out.println(name + " sends: " + message);
11.        mediator.sendMessage(message, this);
12.    }
13.
14.    @Override
15.    public void receive(String message
16.                        , String senderName) {
17.        System.out.println(name + " receives from "
18.                            + senderName + ": " + message);
19.    }
20.}
```

ผู้ใช้ส่งข้อความผ่าน ChatRoom ChatRoom ส่งข้อความไปยังผู้ใช้คนอื่นที่อยู่ในห้อง ยกเว้นผู้ส่งข้อความ

```
1. // Main Class
2. public class MediatorChatExample {
3.     public static void main(String[] args) {
4.
5.         ChatMediator chatRoom = new ChatRoom();
6.
7.         User user1 = new ChatUser(chatRoom, "วาสนา");
8.         User user2 = new ChatUser(chatRoom, "วารุณี");
9.         User user3 = new ChatUser(chatRoom, "วิมล");
10.
11.        chatRoom.addUser(user1);
12.        chatRoom.addUser(user2);
13.        chatRoom.addUser(user3);
14.
15.        // Simulate chat
16.        user1.send("สวัสดีทุกคน");
17.        user2.send("สวัสดีวาสนา");
18.        user3.send("อรุณสวัสดิ์");
19.    }
20.}
```

จะเห็นว่าข้อดีของ Mediator Pattern ในตัวอย่างนี้ได้แก่

- ลด Coupling ผู้ใช้แต่ละคนไม่ต้องเชื่อมต่อกันโดยตรง แต่สื่อสารผ่าน ChatRoom

- ง่ายต่อการเพิ่มผู้ใช้ สามารถเพิ่มผู้ใช้ใหม่ได้โดยไม่กระทบกับผู้ใช้อื่น
- ขยายได้ง่าย สามารถเพิ่มฟีเจอร์ใหม่ เช่น การกรองข้อความ หรือการส่งข้อความส่วนตัว โดยแก้ไขเฉพาะ ChatRoom

ตัวอย่างที่ 2 ระบบควบคุมการจราจรทางอากาศ (Air Traffic Control System) ในระบบควบคุมการจราจรทางอากาศ เครื่องบินหลายลำต้องสื่อสารกันเพื่อหลีกเลี่ยงการชนกัน แต่ถ้าทุกเครื่องบินสื่อสารกันโดยตรง ระบบจะซับซ้อนมาก การใช้ Mediator อย่าง "ศูนย์ควบคุมการบิน (Control Tower)" ช่วยให้ทุกเครื่องบินสื่อสารผ่านตัวกลางได้อย่างมีประสิทธิภาพ

```

1. // Mediator Interface
2. interface AirTrafficControl {
3.     void registerAircraft(Aircraft aircraft);
4.     void sendLandingRequest(Aircraft aircraft);
5. }

```

```

1. // Concrete Mediator
2. class ControlTower implements AirTrafficControl {
3.
4.     private List<Aircraft> aircraftList = new ArrayList<>();
5.
6.     @Override
7.     public void registerAircraft(Aircraft aircraft) {
8.         aircraftList.add(aircraft);
9.     }
10.
11.    @Override
12.    public void sendLandingRequest(Aircraft aircraft) {
13.        for (Aircraft otherAircraft : aircraftList) {
14.            if (otherAircraft != aircraft
15.                && otherAircraft.isLanding()) {
16.                System.out.println(
17.                    "Control Tower: Landing request denied for "
18.                    + aircraft.getName()
19.                    + " due to another aircraft landing.");
20.                return;
21.            }
22.        }
23.        System.out.println(
24.            "Control Tower: Landing request approved for "
25.            + aircraft.getName());
26.        aircraft.land();
27.    }
28.}

```

```

1. // Abstract Aircraft
2. abstract class Aircraft {

```

```

3.
4.     protected AirTrafficControl controlTower;
5.     protected String name;
6.     private boolean landing;
7.
8.     public Aircraft(AirTrafficControl controlTower
9.                     , String name) {
10.         this.controlTower = controlTower;
11.         this.name = name;
12.         this.landing = false;
13.     }
14.
15.     public String getName() {
16.         return name;
17.     }
18.
19.     public boolean isLanding() {
20.         return landing;
21.     }
22.
23.     public void requestLanding() {
24.         controlTower.sendLandingRequest(this);
25.     }
26.
27.     public void land() {
28.         this.landing = true;
29.         System.out.println(name + " is landing.");
30.     }
31.}

```

```

1. // Concrete Aircraft
2. class Plane extends Aircraft {
3.     public Plane(AirTrafficControl controlTower
4.                 , String name) {
5.         super(controlTower, name);
6.     }
7. }

```

```

1. // Main Class
2. public class MediatorAirTrafficControlExample {
3.     public static void main(String[] args) {
4.
5.         ControlTower controlTower = new ControlTower();
6.
7.         Aircraft plane1 = new Plane(controlTower
8.                                     , "Flight A123");
9.         Aircraft plane2 = new Plane(controlTower
10.                                    , "Flight B456");
11.        Aircraft plane3 = new Plane(controlTower
12.                                    , "Flight C789");
13.
14.        controlTower.registerAircraft(plane1);
15.        controlTower.registerAircraft(plane2);

```

```

16.         controlTower.registerAircraft(plane3);
17.
18.         // Simulate landing requests
19.         plane1.requestLanding();
20.         plane2.requestLanding();
21.         plane3.requestLanding();
22.     }
23. }

```

ผลลัพธ์

Control Tower: Landing request approved for Flight A123

Flight A123 is landing.

Control Tower: Landing request denied for Flight B456 due to another aircraft landing.

Control Tower: Landing request denied for Flight C789 due to another aircraft landing.

ตัวอย่างที่ 3 การจัดการการสื่อสารระหว่างตัวละครในเกม (Game Characters) ในเกม RPG หรือเกมแบบมัลติเพลเยอร์ มีตัวละครหลายตัวที่ต้องสื่อสารหรือมีปฏิสัมพันธ์กัน เช่น การโจมตี การรักษา หรือการส่งข้อความหากัน การใช้ Mediator Pattern ช่วยจัดการการสื่อสารเหล่านี้ผ่านตัวกลางอย่าง "GameMediator" เพื่อให้โค้ดอ่านง่าย และสามารถขยายได้ง่ายขึ้น

ปัญหาในการสื่อสารระหว่างตัวละครในเกม

- ตัวละครแต่ละตัวต้องสื่อสารกัน เช่น ตัวละคร A โจมตีตัวละคร B หรือตัวละคร C ใช้สกิลรักษาตัวละครอื่น
- ถ้าตัวละครแต่ละตัวสื่อสารกันโดยตรง โค้ดจะยุ่งเหยิงและยากต่อการจัดการ

```

1. // Mediator Interface
2. interface GameMediator {
3.     void notify(Character sender, String event, Character
        target);
4. }

```

```

1. // Concrete Mediator
2. class GameController implements GameMediator {
3.     @Override
4.     public void notify(Character sender, String event
5.         , Character target) {
6.         if (event.equals("attack")) {

```

```

7.         System.out.println(sender.getName() + " attacks "
8.             + target.getName() + "!");
9.         target.takeDamage(10);
10.    } else if (event.equals("heal")) {
11.        System.out.println(sender.getName() + " heals "
12.            + target.getName() + "!");
13.        target.heal(15);
14.    }
15. }
16.}

```

```

1. // Abstract Character
2. abstract class Character {
3.
4.     protected GameMediator mediator;
5.     protected String name;
6.     protected int health;
7.
8.     public Character(GameMediator mediator, String name
9.         , int health) {
10.        this.mediator = mediator;
11.        this.name = name;
12.        this.health = health;
13.    }
14.
15.    public String getName() {
16.        return name;
17.    }
18.
19.    public int getHealth() {
20.        return health;
21.    }
22.
23.    public void attack(Character target) {
24.        mediator.notify(this, "attack", target);
25.    }
26.
27.    public void heal(Character target) {
28.        mediator.notify(this, "heal", target);
29.    }
30.
31.    public void takeDamage(int damage) {
32.        health -= damage;
33.        System.out.println(name + " takes " + damage
34.            + " damage. Current health: " + health);
35.    }
36.
37.    public void heal(int amount) {
38.        health += amount;
39.        System.out.println(name + " recovers " + amount
40.            + " health. Current health: " + health);
41.    }
42.}

```

```

1. // Concrete Characters
2. class Warrior extends Character {
3.     public Warrior(GameMediator mediator, String name) {
4.         super(mediator, name, 100);
5.     }
6. }

1. class Mage extends Character {
2.     public Mage(GameMediator mediator, String name) {
3.         super(mediator, name, 80);
4.     }
5. }

1. class Healer extends Character {
2.     public Healer(GameMediator mediator, String name) {
3.         super(mediator, name, 70);
4.     }
5. }

1. // Main Class
2. public class MediatorGameExample {
3.     public static void main(String[] args) {
4.
5.         GameController gameController = new GameController();
6.
7.         Character warrior = new Warrior(gameController
8.                                         , "Warrior");
9.         Character mage = new Mage(gameController, "Mage");
10.        Character healer = new Healer(gameController
11.                                     , "Healer");
12.
13.        // Simulate game actions
14.        warrior.attack(mage);
15.        mage.attack(warrior);
16.        healer.heal(warrior);
17.    }
18.}

```

ผลลัพธ์

Warrior attacks Mage!

Mage takes 10 damage. Current health: 70

Mage attacks Warrior!

Warrior takes 10 damage. Current health: 90

Healer heals Warrior!

Warrior recovers 15 health. Current health: 105

อธิบายผลลัพธ์

- Warrior โจมตี Mage
Warrior เรียก mediator.notify เพื่อแจ้งเหตุการณ์โจมตี Mage
GameController รับเหตุการณ์และคำนวณความเสียหายให้ Mage ลดพลังชีวิตลง 10 หน่วย
Mage โจมตี Warrior กระบวนการเดียวกัน แต่เป้าหมายเป็น Warrior
Healer รักษา Warrior
Healer เรียก mediator.notify เพื่อแจ้งเหตุการณ์การรักษา
GameController เพิ่มพลังชีวิตให้ warrior 15 หน่วย

จะเห็นว่า Mediator Pattern ช่วยจัดการการสื่อสารระหว่างตัวละครในเกมได้อย่างมีประสิทธิภาพ

- ลด Coupling ตัวละครไม่ต้องสื่อสารกันโดยตรง
- ง่ายต่อการเพิ่มความสามารถใหม่ เช่น เพิ่มสกิลใหม่ใน Mediator โดยไม่ต้องแก้โค้ดตัวละคร
- ขยายได้ง่าย เพิ่มตัวละครใหม่ เช่น Archer หรือ Tank โดยไม่กระทบกับตัวละครเดิม